

Historias de Usuario

Core E-Commerce Checkout con Sistema de Descuentos Acumulativos — Angular + NestJS (TypeScript), persistencia en memoria

Índice de tickets

Base del proyecto

- [HU-00](#) Documentación inicial y contexto del proyecto
- [HU-01](#) Estructura base del monorepo
- [HU-02](#) Contratos compartidos de tipado

Backend — Catálogo y persistencia

- [HU-03](#) Repositorio de productos en memoria y datos semilla
- [HU-04](#) Consulta del catálogo de productos

Backend — Motor de descuentos

- [HU-05](#) Núcleo del motor de descuentos acumulativos
- [HU-06](#) Regla de descuento por categoría
- [HU-07](#) Regla de descuento por volumen
- [HU-08](#) Regla de descuento por cupón
- [HU-09](#) Límite absoluto de descuento del 35%

Backend — Checkout

- [HU-10](#) Validación de payloads y estructura estándar de error
- [HU-11](#) Cotización del carrito
- [HU-12](#) Validación de stock disponible
- [HU-13](#) Procesamiento y persistencia de la orden
- [HU-14](#) Consulta de órdenes persistidas

Frontend — Carrito

- [HU-15](#) Listado de productos del catálogo
- [HU-16](#) Gestión reactiva del carrito
- [HU-17](#) Control de stock en la interfaz

Frontend — Checkout

- [HU-18](#) Aplicación de cupón y desglose de descuentos
- [HU-19](#) Alerta de límite máximo de ahorro alcanzado
- [HU-20](#) Confirmación de compra
- [HU-21](#) Manejo de errores del backend en la interfaz
- [HU-22](#) Diseño responsivo

Calidad

- [HU-23](#) Cobertura de pruebas del backend y casos de borde
- [HU-24](#) Cobertura de pruebas del frontend y casos de borde

DESCRIPCIÓN

Como equipo de desarrollo, quiero contar con la documentación de contexto y diseño del proyecto antes de implementar cualquier funcionalidad, para que cada sesión de trabajo —humana o asistida por IA— parta de las mismas decisiones de arquitectura, contratos y convenciones, y cada historia se desarrolle alineada con ese contexto.

CRITERIOS FUNCIONALES

- Existe un archivo `CLAUDE.md` en la raíz del repositorio que describe el propósito del proyecto, el stack, la estructura de carpetas objetivo, los patrones de diseño acordados, las decisiones de contrato ya tomadas y la referencia a `docs/historias-usuario.pdf` como fuente de verdad de cada historia.
- Existe `docs/arquitectura.md` en versión inicial que responde: por qué Angular + NestJS + persistencia en memoria, qué trade-offs se asumen, cómo se aísla el motor de descuentos de la persistencia y los controladores, y qué patrones de diseño se implementarán y dónde.
- Existe `docs/ia.md` como esqueleto con las secciones: skills/prompts automatizados, agentes configurados y bitácora de co-creación (porcentaje de código sugerido por IA, lógica crítica manual, sugerencias rechazadas o corregidas), listas para completarse de forma incremental durante el desarrollo.
- Existe `README.md` inicial con el nombre del proyecto, el stack, la descripción del problema y la estructura de carpetas; los comandos se completan cuando exista el scaffold.
- El documento `docs/historias-usuario.pdf` queda versionado en el repositorio.

CRITERIOS TÉCNICOS

- `CLAUDE.md` documenta explícitamente: gestor de paquetes `pnpm` con workspaces, Jest como runner en ambas aplicaciones, umbral de cobertura del 80%, capas del backend (`domain` , `application` , `infrastructure` , `presentation`) y organización del frontend (`components` , `services` , `state` , `interface`).
- `CLAUDE.md` registra las decisiones de diseño cerradas: Strategy + Factory para el motor de descuentos con el tope del 35% como última estrategia de la cadena; Observer con signals para el estado del carrito; cupón inválido tolerado en la cotización y rechazado en el pago; cálculo autoritativo siempre en el servidor; fechas como unix timestamp UTC.
- `CLAUDE.md` define el vocabulario del dominio a usar en identificadores (producto, ítem de carrito, cotización, desglose, orden, cupón, stock) y los nombres de las clases principales previstas en las historias.
- `docs/arquitectura.md` incluye un diagrama textual del flujo de una petición de checkout a través de las capas y la ubicación de cada patrón.
- La documentación se redacta en español y se mantiene como documento vivo: cada historia posterior que modifique una decisión debe actualizarla en el mismo cambio.

CRITERIOS DE SEGURIDAD

- La documentación no contiene credenciales, tokens, rutas absolutas de la máquina de desarrollo ni datos personales.
- `docs/arquitectura.md` declara los principios de seguridad transversales que las historias deben cumplir: validación de entrada en el borde, cálculo de precios y descuentos exclusivamente en el servidor, sin exposición de detalles internos en errores, sin datos sensibles en logs.

DESCRIPCIÓN

Como equipo de desarrollo, quiero contar con la estructura base del monorepo con las aplicaciones de frontend y backend inicializadas, para que todo el trabajo posterior se construya sobre una base consistente que arranque y ejecute pruebas desde el primer momento.

CRITERIOS FUNCIONALES

- El repositorio contiene las carpetas `apps/backend`, `apps/frontend`, `packages`, `docs` y un `README.md` en la raíz.
- El backend arranca y expone un endpoint `GET /health` que responde `200` con el estado del servicio.
- El frontend arranca y muestra una pantalla inicial vacía del módulo de checkout.
- Un único comando desde la raíz levanta ambas aplicaciones; otro ejecuta las pruebas de ambas con reporte de cobertura.
- El `README.md` creado en HU-00 se actualiza con los comandos reales de instalación, arranque y pruebas.

CRITERIOS TÉCNICOS

- Gestor de paquetes `pnpm` con workspaces para `apps/*` y `packages/*`; versión de Node fijada mediante `.nvmrc` y `engines`.
- Backend generado con NestJS CLI; `tsconfig` con `strict`, `noImplicitAny`, `strictNullChecks` y `noUnusedLocals` habilitados.
- Frontend generado con Angular CLI en modo `strict`, componentes standalone, estilos SCSS y enrutamiento.
- Jest como runner de pruebas en ambas aplicaciones con `coverageThreshold` global del 80% en statements, branches, functions y lines.
- Estructura de carpetas por capas creada en el backend (`domain`, `application`, `infrastructure`, `presentation`) y por responsabilidad en el frontend (`components`, `services`, `state`, `interface`).
- `.editorconfig` y configuración de lint compartida (2 espacios, comillas simples, punto y coma obligatorio).

CRITERIOS DE SEGURIDAD

- `.gitignore` excluye `node_modules`, `dist`, `coverage`, `.angular` y cualquier archivo `.env`.
- Se versiona únicamente `.env.example` con las variables `PORT` y `CORS_ORIGIN`, sin valores sensibles.
- El backend habilita CORS restringido al origen del frontend configurado por variable de entorno, nunca con comodín.
- Ninguna dependencia instalada presenta vulnerabilidades críticas conocidas en el momento de la instalación (`pnpm audit`).

DESCRIPCIÓN

Como equipo de desarrollo, quiero un paquete compartido con las interfaces, DTOs y enumerables del dominio de checkout, para que frontend y backend utilicen exactamente los mismos contratos y el tipado sea estricto de extremo a extremo.

CRITERIOS FUNCIONALES

- Existe el paquete `packages/shared` consumible desde ambas aplicaciones mediante el workspace.
- Define los contratos de producto, ítem del carrito, solicitud de cotización, solicitud de checkout, desglose de descuentos, orden y estructura de error.
- Define los enumerables de categoría de producto y de tipo de descuento.
- Cualquier cambio en un contrato provoca error de compilación en la aplicación que no lo respete.

CRITERIOS TÉCNICOS

- Interfaces con prefijo `I` y PascalCase (`IProduct` , `ICartItem` , `ICheckoutRequest` , `IDiscountBreakdown` , `IOrder` , `IApiError`).
- Enumerables con sufijo `Enum` y miembros en `UPPER_SNAKE_CASE` (`ProductCategoryEnum` , `DiscountTypeEnum`).
- El paquete no depende de Angular ni de NestJS; solo TypeScript puro.
- Los montos monetarios se tipan como `number` con precisión de dos decimales documentada en el contrato; los porcentajes como fracción decimal (0.10, 0.05, 0.15, 0.35).
- Constantes del dominio centralizadas: porcentajes de cada regla, umbral de volumen (100) y tope máximo (0.35).

CRITERIOS DE SEGURIDAD

- El contrato de respuesta no expone información interna del servidor (rutas, stack traces, nombres de clases).
- El contrato de solicitud de checkout recibe únicamente identificadores y cantidades; los precios nunca viajan desde el cliente hacia el servidor.

DESCRIPCIÓN

Como API de la tienda, quiero disponer de un repositorio de productos en memoria inicializado con un catálogo pre-configurado, para que el resto de casos de uso consulten y actualicen el stock sin depender de una base de datos externa.

CRITERIOS FUNCIONALES

- Al iniciar la aplicación se cargan al menos 6 productos con id, nombre, precio unitario, categoría y stock inicial.
- El catálogo semilla permite reproducir todos los escenarios de la demo: al menos dos productos de categoría Tecnología (uno de alto valor), productos de otras categorías, un producto con stock bajo (1 o 2 unidades) y combinaciones cuyo descuento acumulado supere el 35%.
- El repositorio permite obtener todos los productos, obtener uno por id y decrementar el stock de un producto.
- Al reiniciar el servidor el catálogo vuelve a su estado semilla.

CRITERIOS TÉCNICOS

- Puerto abstracto `IProductRepository` definido en la capa de dominio; implementación `InMemoryProductRepository` en infraestructura, registrada mediante token de inyección.
- Los casos de uso dependen únicamente del puerto, nunca de la implementación concreta.
- Los datos semilla viven en un archivo dedicado de infraestructura, separado de la implementación del repositorio.
- El repositorio devuelve copias de los productos, no referencias internas mutables.
- Pruebas unitarias del repositorio: lectura, búsqueda por id existente e inexistente, decremento de stock exitoso y rechazo cuando el stock es insuficiente.

CRITERIOS DE SEGURIDAD

- El decremento de stock nunca produce valores negativos; ante cantidad mayor al disponible la operación se rechaza sin modificar estado.
- La operación de decremento es atómica por producto dentro del proceso: se valida y actualiza en un mismo paso para evitar condiciones de carrera entre peticiones concurrentes.

DESCRIPCIÓN

Como cliente de la tienda, quiero consultar el catálogo de productos con su stock actual, para poder decidir qué agregar al carrito.

CRITERIOS FUNCIONALES

- `GET /products` responde `200` con el arreglo de productos: id, nombre, precio unitario, categoría y stock disponible.
- El stock reflejado es el actual, es decir, ya descontadas las compras procesadas previamente.
- Si el catálogo estuviera vacío responde `200` con arreglo vacío, nunca error.

CRITERIOS TÉCNICOS

- Controlador en capa de presentación que delega en el caso de uso `GetProductsUseCase`; el controlador no accede al repositorio directamente.
- La respuesta cumple el contrato `IProduct[]` del paquete compartido.
- Pruebas unitarias del caso de uso y del controlador con el repositorio mockeado.

CRITERIOS DE SEGURIDAD

- El endpoint es de solo lectura; no acepta parámetros que modifiquen estado.
- No se exponen campos internos del producto que no formen parte del contrato público.

DESCRIPCIÓN

Como API de la tienda, quiero un motor de descuentos que ejecute reglas en cascada con orden de precedencia configurable, para calcular de forma consistente el desglose de descuentos de cualquier carrito sin acoplarse a la persistencia ni a la capa HTTP.

CRITERIOS FUNCIONALES

- El motor recibe los ítems del carrito (con precio y categoría resueltos desde el catálogo) y un cupón opcional, y devuelve el desglose completo: subtotal original, monto de cada descuento aplicado, total de descuento, porcentaje efectivo, ahorro total y total final a pagar.
- Las reglas se aplican de forma secuencial y multiplicativa: cada regla opera sobre el resultado de la anterior, nunca sobre el subtotal original de forma aditiva.
- Con un carrito sin reglas aplicables el total final es igual al subtotal original y todos los descuentos son cero.
- Con un carrito vacío el motor devuelve todos los valores en cero sin lanzar excepción.

CRITERIOS TÉCNICOS

- Patrón **Strategy**: interfaz `IDiscountStrategy` con un método que recibe el contexto de cálculo y devuelve el contexto actualizado; cada regla de negocio es una implementación independiente.
- Patrón **Factory**: `DiscountStrategyFactory` construye la cadena ordenada de estrategias según la precedencia definida; el motor no conoce las clases concretas.
- El motor vive en la capa de dominio: sin decoradores de NestJS, sin imports de infraestructura, sin efectos secundarios; es una función pura respecto de sus entradas.
- Los cálculos monetarios se redondean a dos decimales solo al construir el desglose final; los pasos intermedios conservan precisión completa.
- El contexto de cálculo es un objeto inmutable tipado; cada estrategia devuelve una nueva instancia.
- Pruebas unitarias del motor con estrategias mockeadas verificando el orden de ejecución y la acumulación en cascada.

CRITERIOS DE SEGURIDAD

- El motor es la única fuente de verdad del cálculo; ningún total calculado en el cliente se acepta ni se reutiliza.
- Los precios que alimentan el motor provienen exclusivamente del repositorio de productos del servidor.

DESCRIPCIÓN

Como área de negocio, quiero que los productos de la categoría Tecnología reciban un 10% de descuento sobre su propio precio, para incentivar la compra de esa categoría.

CRITERIOS FUNCIONALES

- Si el carrito contiene al menos un ítem de categoría Tecnología, se aplica 10% únicamente sobre el valor de esos ítems (precio unitario × cantidad).
- Los ítems de otras categorías no se ven afectados por esta regla.
- Si no hay ítems de Tecnología el descuento por categoría es cero y el subtotal resultante es idéntico al original.
- El monto del descuento por categoría aparece identificado en el desglose.

CRITERIOS TÉCNICOS

- Implementación `CategoryDiscountStrategy` de `IDiscountStrategy`, primera en la cadena de la Factory.
- El porcentaje y la categoría objetivo provienen de las constantes del paquete compartido, no de literales en la clase.
- Pruebas unitarias: carrito solo con Tecnología, carrito mixto, carrito sin Tecnología, ítem de Tecnología con cantidad mayor a uno.

CRITERIOS DE SEGURIDAD

- La categoría se compara contra el enumerable del dominio; valores fuera del enumerable no activan la regla.

DESCRIPCIÓN

Como área de negocio, quiero que las compras cuyo subtotal supere los \$100 USD reciban un 5% de descuento adicional sobre todo el carrito, para incentivar compras de mayor valor.

CRITERIOS FUNCIONALES

- La regla evalúa el subtotal resultante después del descuento por categoría, no el subtotal original.
- Se aplica solo si ese subtotal es estrictamente mayor a 100; un subtotal exactamente igual a 100 no activa la regla.
- El 5% se aplica sobre el total completo del carrito tras la regla anterior.
- El monto del descuento por volumen aparece identificado en el desglose.

CRITERIOS TÉCNICOS

- Implementación `VolumeDiscountStrategy`, segunda en la cadena de la Factory.
- Umbral y porcentaje provienen de las constantes del paquete compartido.
- Pruebas unitarias: subtotal por debajo del umbral, exactamente en el umbral, por encima del umbral, y caso donde el descuento por categoría hace que el subtotal caiga por debajo de 100 aunque el original lo superaba.

CRITERIOS DE SEGURIDAD

- La comparación contra el umbral se realiza sobre valores calculados en el servidor, nunca sobre un subtotal recibido del cliente.

DESCRIPCIÓN

Como cliente, quiero aplicar el cupón promocional `WELCOME2026` para obtener un 15% de descuento adicional sobre el total de mi compra tras los descuentos anteriores.

CRITERIOS FUNCIONALES

- Si el cupón enviado corresponde a un cupón activo, se aplica 15% sobre el total obtenido después de la regla de volumen.
- Si no se envía cupón, la regla no aplica y el desglose muestra descuento por cupón en cero.
- Si el cupón no existe o está inactivo, la regla no aplica y el resultado indica que el cupón es inválido para que la capa superior decida cómo responder.
- El monto del descuento por cupón aparece identificado en el desglose.

CRITERIOS TÉCNICOS

- Implementación `CouponDiscountStrategy`, tercera en la cadena de la Factory.
- Los cupones activos se resuelven a través de un puerto `ICouponRepository` con implementación en memoria; la estrategia no contiene el código del cupón como literal.
- El modelo de cupón incluye código, porcentaje y estado activo, dejando extensible la fecha de expiración.
- Pruebas unitarias: cupón válido, cupón inexistente, cupón inactivo, sin cupón, cupón con espacios o minúsculas.

CRITERIOS DE SEGURIDAD

- La validación del cupón ocurre exclusivamente en el servidor; el cliente nunca recibe el listado de cupones válidos.
- El código de cupón se normaliza (trim y mayúsculas) y se limita en longitud antes de compararlo.
- Los mensajes de error no revelan si el cupón existió alguna vez ni pistas sobre cupones activos.

DESCRIPCIÓN

Como área de negocio, quiero que el descuento total consolidado nunca supere el 35% del valor original de los productos, para proteger el margen de ganancia de la tienda.

CRITERIOS FUNCIONALES

- Si el descuento acumulado tras las tres reglas supera el 35% del subtotal original, el descuento total se trunca exactamente al 35% y el total final se calcula como subtotal original $\times$ 0.65.
- Si el descuento acumulado es igual o inferior al 35%, no se altera.
- El desglose incluye un indicador booleano de que se alcanzó el límite máximo, además del porcentaje efectivo aplicado (que será exactamente 35% en ese caso).
- Los descuentos individuales del desglose se mantienen informativos; el desglose deja explícito el monto de ajuste por tope aplicado.

CRITERIOS TÉCNICOS

- Implementación `MaxDiscountCapStrategy` como última etapa de la cadena de la Factory, garantizando que siempre se ejecute después de las reglas de descuento.
- El porcentaje máximo proviene de las constantes del paquete compartido.
- La comparación se realiza con tolerancia de precisión de punto flotante para que un 35% exacto no se considere superado.
- Pruebas unitarias: descuento acumulado por debajo del tope, exactamente en el tope, por encima del tope (caso real: Tecnología + volumen + cupón $\approx$ 27.3%, y combinaciones que lo superen), y verificación de que el total final nunca es menor al 65% del original.

CRITERIOS DE SEGURIDAD

- El tope se aplica en el servidor en cada cálculo; no existe forma de omitir la etapa desde la petición.

DESCRIPCIÓN

Como API de la tienda, quiero validar la estructura de toda petición entrante y responder los errores con un formato uniforme, para rechazar datos corruptos antes de ejecutar lógica de negocio y facilitar su manejo en el frontend.

CRITERIOS FUNCIONALES

- Toda respuesta de error tiene la forma `{ error: { code, message } }`, con código legible por máquina y mensaje legible por humano en español.
- Se responde `400` ante: cuerpo vacío, carrito vacío, ítem sin id, id con formato inválido, cantidad no entera, cantidad menor o igual a cero, cupón que no sea texto, campos desconocidos.
- Se responde `404` cuando un id de producto no existe en el catálogo.
- Se responde `500` con mensaje genérico ante errores no controlados, sin detalles internos.

CRITERIOS TÉCNICOS

- DTOs con `class-validator` y `class-transformer`; `ValidationPipe` global con `whitelist`, `forbidNonWhitelisted` y `transform`.
- Filtro global de excepciones que traduce excepciones de dominio (`BaseError` y derivadas) y de Nest al formato estándar.
- Códigos de error con prefijo de proyecto y consecutivo por capa (1xxx presentación, 2xxx aplicación, 3xxx dominio), centralizados en un enumerable.
- Pruebas unitarias del filtro de excepciones y pruebas de validación de cada DTO con casos válidos e inválidos.

CRITERIOS DE SEGURIDAD

- Los stack traces nunca llegan al cliente; se registran solo en el servidor.
- Se limita el tamaño del cuerpo de la petición y la cantidad máxima de ítems por carrito.
- Las propiedades no declaradas en el DTO se rechazan, evitando inyección de campos.
- Los logs de error registran únicamente el mensaje, nunca el objeto de error completo ni el payload del usuario.

DESCRIPCIÓN

Como cliente, quiero obtener el desglose de descuentos de mi carrito actual con o sin cupón, sin comprometer stock ni generar una orden, para revisar cuánto pagaría antes de confirmar la compra.

CRITERIOS FUNCIONALES

- `POST /checkout/quote` recibe los ítems (id y cantidad) y un cupón opcional, y responde `200` con el desglose completo de descuentos.
- La cotización no modifica el stock ni persiste ninguna orden.
- Si el cupón es inválido, la cotización se calcula igualmente sin la regla de cupón y la respuesta indica que el cupón no es válido, para que el frontend lo informe.
- La respuesta incluye el indicador de límite máximo alcanzado.
- Ante carrito vacío o ítems inválidos aplica la validación de HU-10.

CRITERIOS TÉCNICOS

- Caso de uso `QuoteCartUseCase` que resuelve productos desde el repositorio, arma el contexto de cálculo e invoca el motor de descuentos.
- El controlador delega en el caso de uso; no contiene lógica de negocio ni acceso a repositorios.
- La respuesta cumple el contrato `IDiscountBreakdown` del paquete compartido.
- Pruebas unitarias del caso de uso con repositorio y motor mockeados; pruebas del controlador.

CRITERIOS DE SEGURIDAD

- La cotización recalcula todo desde el catálogo del servidor; ignora cualquier precio o total que pudiera venir en la petición.
- Al no mutar estado, el endpoint es idempotente y seguro ante reintentos.

DESCRIPCIÓN

Como API de la tienda, quiero verificar que cada producto del carrito tenga stock suficiente antes de procesar una orden, para no vender unidades que no existen.

CRITERIOS FUNCIONALES

- Antes de calcular y persistir una orden, se compara la cantidad solicitada de cada ítem con el stock actual del producto.
- Si algún ítem excede el stock, la orden se rechaza con `409` indicando el producto afectado, la cantidad solicitada y la disponible.
- Si varios ítems exceden el stock, la respuesta lista todos, no solo el primero.
- Ítems repetidos con el mismo id se consolidan sumando cantidades antes de validar.

CRITERIOS TÉCNICOS

- Servicio de dominio `StockValidator` puro, sin dependencias de framework, que recibe ítems y productos y devuelve el listado de conflictos.
- Excepción de dominio `InsufficientStockError` derivada de `BaseError`, traducida a `409` por el filtro de HU-10.
- Pruebas unitarias: stock suficiente, stock exacto, stock insuficiente en uno y en varios ítems, ítems duplicados consolidados, producto con stock cero.

CRITERIOS DE SEGURIDAD

- La validación se realiza contra el estado del servidor en el momento de la petición, nunca contra el stock que el cliente cree tener.
- La validación y el decremento posterior se ejecutan sin ceder el control entre ambos pasos para evitar sobreventa por concurrencia.

DESCRIPCIÓN

Como API de la tienda, quiero procesar una petición de checkout validando stock, recalculando descuentos, decrementando inventario y persistiendo la orden, para entregar al cliente una confirmación con totales exactos y consistentes.

CRITERIOS FUNCIONALES

- `POST /checkout` recibe los ítems y un cupón opcional, y responde `201` con el id de la orden generada y el desglose completo de totales.
- Si el cupón enviado es inválido, la orden se rechaza con `400` y no se persiste nada.
- Si hay stock insuficiente, la orden se rechaza con `409` y no se persiste nada ni se modifica el stock.
- Al procesarse con éxito, el stock de cada producto se decrementa en la cantidad comprada y una consulta posterior al catálogo lo refleja.
- La orden persistida contiene id, fecha de creación, ítems con precio unitario al momento de la compra, cupón aplicado, desglose de descuentos y total final.

CRITERIOS TÉCNICOS

- Caso de uso `ProcessCheckoutUseCase` que orquesta: resolución de productos, validación de stock, motor de descuentos, decremento de stock y persistencia; no contiene reglas matemáticas propias.
- Puerto `IOrderRepository` en dominio con implementación `InMemoryOrderRepository` en infraestructura.
- El id de orden se genera en el servidor (UUID v4).
- La fecha de creación se almacena como unix timestamp en UTC.
- Orden de ejecución garantizado: validar todo primero, mutar estado después; si cualquier paso falla antes de persistir, ningún stock queda decrementado.
- Pruebas unitarias del caso de uso con todos los puertos mockeados: flujo exitoso, cupón inválido, stock insuficiente, verificación de que el repositorio de órdenes recibe exactamente el desglose calculado por el motor.

CRITERIOS DE SEGURIDAD

- Los totales de la orden son los calculados por el servidor; cualquier total presente en la petición se descarta.
- El id de orden no es secuencial ni predecible.
- El log de la operación registra id de orden y total, nunca el payload completo del cliente.

DESCRIPCIÓN

Como operador de la tienda, quiero consultar las órdenes procesadas, para confirmar que la persistencia funciona y auditar los totales calculados.

CRITERIOS FUNCIONALES

- `GET /orders` responde `200` con el listado de órdenes persistidas ordenadas de más reciente a más antigua.
- `GET /orders/:id` responde `200` con la orden completa, o `404` si no existe.
- Cada orden incluye el desglose de descuentos tal como se calculó en el momento de la compra.

CRITERIOS TÉCNICOS

- Casos de uso `GetOrdersUseCase` y `GetOrderByIdUseCase` que dependen del puerto `IOrderRepository`.
- El parámetro `id` se valida como UUID mediante pipe antes de llegar al caso de uso.
- Pruebas unitarias de casos de uso y controlador, incluyendo el caso de id inexistente e id con formato inválido.

CRITERIOS DE SEGURIDAD

- Un id con formato inválido responde `400`, no `500`.
- El endpoint es de solo lectura y no expone datos internos más allá del contrato `IOrder`.

DESCRIPCIÓN

Como cliente de la tienda, quiero ver la lista de productos disponibles con su precio, categoría y stock, para elegir cuáles agregar a mi carrito.

CRITERIOS FUNCIONALES

- Al cargar la pantalla se muestra el catálogo obtenido del backend con id, nombre, precio unitario formateado en USD, categoría y stock disponible.
- Mientras se carga el catálogo se muestra un indicador de carga; si la carga falla se muestra un mensaje de error con opción de reintentar.
- Cada producto tiene una acción para agregarlo al carrito.
- Los productos de categoría Tecnología se distinguen visualmente indicando que aplican al descuento de categoría.

CRITERIOS TÉCNICOS

- Servicio `ProductsService` con `HttpClient` tipado con `IProduct[]` del paquete compartido; URL base desde `environment`.
- Componente standalone `ProductListComponent` con estado de carga, error y datos manejado con signals.
- Atributos `data-test` en el contenedor, cada tarjeta de producto y el botón de agregar.
- Pruebas unitarias del servicio (mock de HTTP) y del componente (render de lista, estado de carga, estado de error).

CRITERIOS DE SEGURIDAD

- Los datos del backend se renderizan mediante interpolación de Angular; nunca con `innerHTML`.
- La URL del backend no está hardcodeada en componentes; proviene de la configuración de entorno.

DESCRIPCIÓN

Como cliente de la tienda, quiero agregar, quitar y ajustar cantidades de productos en mi carrito, y ver el subtotal original actualizado de inmediato, para controlar mi compra en tiempo real.

CRITERIOS FUNCIONALES

- Agregar un producto lo incorpora al carrito con cantidad 1; si ya existe, incrementa su cantidad.
- Es posible incrementar, decrementar y eliminar un ítem; al decrementar a cero el ítem se elimina.
- El subtotal original (suma de precio unitario $\times$ cantidad) y la cantidad total de unidades se actualizan al instante ante cualquier cambio, sin llamar al backend.
- Existe una acción para vaciar el carrito completo.
- Cualquier modificación del carrito invalida el desglose de descuentos previamente cotizado hasta que se vuelva a cotizar.

CRITERIOS TÉCNICOS

- Patrón **Observer** implementado con un store `CartStore` basado en signals: estado privado, selectores derivados con `computed` (subtotal, total de unidades, ítems) y métodos de mutación explícitos.
- Los componentes se suscriben a los selectores; ninguno mantiene copia local del estado del carrito.
- El estado del carrito es inmutable: cada mutación produce un nuevo arreglo de ítems.
- El cálculo del subtotal usa aritmética en centavos o redondeo controlado a dos decimales para evitar errores de punto flotante en la interfaz.
- Atributos `data-test` en el contenedor del carrito, cada fila, botones de incrementar, decrementar, eliminar y vaciar, y en el subtotal.
- Pruebas unitarias del store: agregar nuevo, agregar existente, incrementar, decrementar a cero, eliminar, vaciar, subtotal con múltiples ítems, invalidación del desglose.

CRITERIOS DE SEGURIDAD

- El carrito almacena solo id, nombre, precio de referencia y cantidad; nunca se envían precios al backend.
- El subtotal mostrado es informativo; el valor autoritativo siempre proviene de la cotización del servidor.

DESCRIPCIÓN

Como cliente de la tienda, quiero que la interfaz me impida agregar más unidades de las disponibles, para no intentar comprar productos sin stock.

CRITERIOS FUNCIONALES

- El botón de agregar y el de incrementar se deshabilitan cuando la cantidad en el carrito alcanza el stock disponible del producto.
- Un producto con stock cero muestra la etiqueta "Sin stock" y no puede agregarse.
- Al intentar superar el stock se muestra un aviso indicando la cantidad máxima disponible.
- El stock mostrado en el catálogo refleja las unidades restantes después de las que ya están en el carrito.

CRITERIOS TÉCNICOS

- Selector derivado en `CartStore` que expone, por producto, el stock restante disponible para agregar.
- La lógica de "puede agregar" es una función pura reusable, no duplicada entre catálogo y carrito.
- Pruebas unitarias: agregar hasta el límite, intento de superar el límite, producto con stock cero, actualización del stock disponible tras un checkout exitoso.

CRITERIOS DE SEGURIDAD

- El control en la interfaz es una comodidad de experiencia de usuario; la validación autoritativa de stock permanece en el backend (HU-12) y la interfaz maneja su rechazo (HU-21).

DESCRIPCIÓN

Como cliente, quiero ingresar un código de cupón, presionar "Aplicar" y ver el desglose detallado de los descuentos calculados por el servidor, para saber exactamente cuánto ahorro y cuánto pagaré.

CRITERIOS FUNCIONALES

- Existe un campo de texto para el cupón y un botón "Aplicar"; el botón está deshabilitado con el carrito vacío.
- Al aplicar, se solicita la cotización al backend y se muestra: subtotal original, descuento de categoría, descuento por volumen, descuento por cupón, porcentaje de descuento efectivo, ahorro total y valor final a pagar.
- Las líneas de descuento con valor cero se muestran igualmente con "\$0.00" para que el desglose sea siempre completo.
- Si el cupón es inválido, el desglose se muestra sin el descuento de cupón y aparece un mensaje claro "Cupón no válido" junto al campo.
- Es posible cotizar sin cupón para ver los descuentos automáticos.
- Si el carrito cambia después de cotizar, el desglose se marca como desactualizado hasta volver a aplicar.
- Mientras la cotización está en curso se muestra un indicador de carga y el botón se deshabilita.

CRITERIOS TÉCNICOS

- Servicio `CheckoutService` con método de cotización tipado con `ICheckoutRequest` e `IDiscountBreakdown` del paquete compartido.
- El desglose se almacena en `CartStore` como parte del estado observable, no en el componente.
- Componentes standalone `CouponFormComponent` y `DiscountBreakdownComponent`, con entradas tipadas y sin lógica de negocio.
- El campo de cupón usa formularios reactivos con validación de longitud máxima y patrón alfanumérico.
- Atributos `data-test` en el input de cupón, el botón de aplicar, cada línea del desglose y el total final.
- Pruebas unitarias: envío de cotización con y sin cupón, render del desglose completo, mensaje de cupón inválido, estado de carga, invalidación tras cambio en el carrito.

CRITERIOS DE SEGURIDAD

- El frontend no conoce ni valida el código de cupón; solo lo envía al servidor.
- El valor del cupón se normaliza y limita en longitud antes de enviarse.
- Ningún cálculo de descuento se realiza en el cliente; todo el desglose mostrado proviene de la respuesta del servidor.

DESCRIPCIÓN

Como cliente, quiero recibir una notificación visual persistente y distintiva cuando la combinación de mis productos y el cupón alcance el límite del 35% de descuento, para saber que estoy obteniendo el ahorro máximo permitido.

CRITERIOS FUNCIONALES

- Cuando la cotización o la confirmación de compra indica que se alcanzó el límite máximo, se muestra una notificación con el texto exacto: **"¡Enhorabuena! Has alcanzado el límite máximo de ahorro permitido (35%)"**.
- La notificación es persistente: permanece visible mientras el desglose vigente tenga el límite alcanzado; no desaparece por tiempo.
- La notificación es visualmente distintiva respecto de mensajes informativos y de error (color, ícono y posición propios).
- Si el carrito cambia y una nueva cotización ya no alcanza el límite, la notificación desaparece.
- El porcentaje efectivo mostrado en el desglose es exactamente 35% cuando la alerta está activa.

CRITERIOS TÉCNICOS

- Componente standalone `MaxDiscountAlertComponent` cuya visibilidad depende de un selector derivado de `CartStore` (`isMaxDiscountReached`), no de lógica en la plantilla.
- El texto de la alerta se define en una constante única, no repetido en la plantilla y en las pruebas por separado.
- Atributo `data-test` en el contenedor de la alerta.
- Pruebas unitarias: la alerta se muestra cuando el indicador es verdadero, se oculta cuando es falso, contiene el texto exacto, y desaparece al invalidarse el desglose.

CRITERIOS DE SEGURIDAD

- La alerta se dispara únicamente con el indicador provisto por el servidor; el frontend no deduce el límite comparando montos.
- La alerta usa `role="status"` y `aria-live` para ser anunciada por lectores de pantalla.

DESCRIPCIÓN

Como cliente, quiero confirmar mi compra y recibir la confirmación de la orden con sus totales, para completar el proceso de checkout.

CRITERIOS FUNCIONALES

- Existe un botón "Pagar" habilitado solo cuando el carrito tiene ítems y existe una cotización vigente (no desactualizada).
- Al confirmar, se envía la petición de checkout con los ítems y el cupón aplicado; durante el proceso el botón se deshabilita y muestra estado de carga.
- Ante éxito se muestra una pantalla o panel de confirmación con el id de la orden, la fecha, los ítems comprados y el desglose final de totales.
- Tras la confirmación el carrito se vacía, el campo de cupón se limpia y el catálogo se recarga mostrando el stock actualizado.
- Existe una acción para volver al catálogo e iniciar una nueva compra.

CRITERIOS TÉCNICOS

- Método de checkout en `CheckoutService` tipado con `ICheckoutRequest` e `IOrder`.
- Componente standalone `OrderConfirmationComponent` que recibe la orden como entrada tipada.
- La fecha de la orden (unix timestamp) se convierte a zona local con `dayjs` en el componente.
- El botón "Pagar" previene doble envío: se ignoran clics mientras hay una petición en curso.
- Atributos `data-test` en el botón de pagar, el panel de confirmación, el id de orden y el total.
- Pruebas unitarias: envío del checkout con los datos correctos, render de la confirmación, vaciado del carrito y recarga del catálogo tras éxito, prevención de doble envío.

CRITERIOS DE SEGURIDAD

- La petición de checkout envía únicamente ids, cantidades y cupón; nunca precios ni totales.
- Los totales mostrados en la confirmación son los devueltos por el servidor, no los de la cotización previa.
- No se almacena información de la orden en `localStorage` ni en ningún almacenamiento persistente del navegador.

DESCRIPCIÓN

Como cliente, quiero que los errores devueltos por el servidor se me muestren de forma clara y comprensible, para entender qué ocurrió y cómo continuar sin que la aplicación se rompa.

CRITERIOS FUNCIONALES

- Ante `409` por stock insuficiente se muestra un mensaje que identifica el producto afectado y la cantidad disponible, y el catálogo se recarga con el stock real.
- Ante `400` por cupón inválido en el pago se muestra el mensaje junto al campo de cupón y se invalida la cotización.
- Ante `400` por payload inválido o `404` por producto inexistente se muestra un mensaje genérico y se ofrece recargar el catálogo.
- Ante `500` o fallo de red se muestra un mensaje de error general con opción de reintentar.
- Los mensajes de error se distinguen visualmente de la alerta de ahorro máximo y de los mensajes informativos.

CRITERIOS TÉCNICOS

- Interceptor HTTP que normaliza toda respuesta de error al contrato `IApiError` del paquete compartido, incluyendo fallos de red sin respuesta.
- Servicio `NotificationService` centralizado para mostrar mensajes de error e información; los componentes no construyen mensajes por su cuenta.
- Los errores se tipan como `HttpErrorResponse` e `IApiError`; no se usa `any`.
- Atributo `data-test` en el contenedor de notificaciones de error.
- Pruebas unitarias del interceptor (cada código HTTP y fallo de red) y del servicio de notificaciones.

CRITERIOS DE SEGURIDAD

- Los mensajes técnicos del servidor no se muestran textualmente al usuario final salvo los códigos `400` / `409` diseñados para ello; el resto se traduce a mensajes genéricos.
- Los detalles del error se registran en consola solo en modo desarrollo y nunca incluyen el payload enviado.

DESCRIPCIÓN

Como cliente, quiero utilizar la tienda desde un dispositivo móvil o de escritorio con una interfaz que se adapte al tamaño de pantalla, para completar mi compra cómodamente desde cualquier dispositivo.

CRITERIOS FUNCIONALES

- En escritorio, el catálogo y el panel de carrito/checkout se muestran lado a lado.
- En móvil (ancho menor a 768px), el contenido se apila en una sola columna y el resumen del carrito con el total permanece accesible.
- Botones y controles tienen un área táctil mínima de 44×44px en móvil.
- No aparece desplazamiento horizontal en ningún tamaño de pantalla entre 320px y 1920px.
- La alerta de ahorro máximo y las notificaciones de error son visibles sin desplazamiento en ambos formatos.

CRITERIOS TÉCNICOS

- Layout con CSS Grid y Flexbox; breakpoints definidos como variables SCSS centralizadas.
- Unidades relativas para tipografía y espaciado; imágenes o iconos con `max-width: 100%`.
- Estilos encapsulados por componente; uso de `::ng-deep` únicamente precedido de `:host`.
- Verificación manual en los anchos 320, 375, 768, 1024 y 1440px documentada como parte de la aceptación.

CRITERIOS DE SEGURIDAD

- La meta etiqueta `viewport` no deshabilita el zoom del usuario (`user-scalable` no se fija en `no`), preservando accesibilidad.

DESCRIPCIÓN

Como equipo de desarrollo, quiero que el backend cuente con una suite de pruebas unitarias que supere el 80% de cobertura y cubra explícitamente los casos de borde del motor de descuentos y la validación de stock, para garantizar la confiabilidad del cálculo y detectar regresiones.

CRITERIOS FUNCIONALES

- La ejecución del comando de cobertura reporta al menos 80% en statements, branches, functions y lines, y falla si cualquiera baja del umbral.
- Existen pruebas nombradas explícitamente para cada caso de borde exigido:
 - Descuento acumulado exactamente en 35% (no se trunca) y superado (se trunca a 35% exacto).
 - Carrito vacío en cotización y en checkout.
 - Carrito con datos corruptos: cantidades negativas, cero, decimales, ids inexistentes, campos faltantes y campos desconocidos.
 - Cupón no registrado, cupón inactivo, cupón vacío.
 - Compra con stock insuficiente en uno y en varios productos; compra que agota exactamente el stock.
- Existe al menos una prueba de integración del flujo completo de checkout (petición HTTP → orden persistida → stock decrementado) con el módulo de Nest real y repositorios en memoria.

CRITERIOS TÉCNICOS

- Mocks en carpeta `mocks/` de cada módulo con sufijo `Mock`; spies con sufijo `Spy`.
- Un `describe` por archivo con el nombre de la clase y el proceso; textos de `it` en español con estructura "debería [resultado] cuando [condición]".
- Toda prueba valida el valor de retorno o el estado resultante, no únicamente que el método se ejecutó.
- Datos de prueba representativos del dominio (nombres de productos, precios reales), sin placeholders sin sentido.
- Las pruebas del motor de descuentos verifican montos exactos con precisión de dos decimales para cada escenario.

CRITERIOS DE SEGURIDAD

- Existen pruebas que verifican que el filtro de excepciones no filtra stack traces ni detalles internos en la respuesta.
- Existen pruebas que verifican que los precios enviados por el cliente son ignorados y se usan los del catálogo.

DESCRIPCIÓN

Como equipo de desarrollo, quiero que el frontend cuente con una suite de pruebas unitarias que supere el 80% de cobertura en el manejo de estado del carrito y la lógica de alertas, para garantizar que la interfaz reaccione correctamente ante todos los escenarios del checkout.

CRITERIOS FUNCIONALES

- La ejecución del comando de cobertura reporta al menos 80% en statements, branches, functions y lines, y falla si cualquiera baja del umbral.
- Existen pruebas nombradas explícitamente para los casos de borde de la interfaz:
 - Alerta de 35% visible con indicador verdadero, oculta con indicador falso, y ocultada al modificar el carrito.
 - Carrito vacío: botones de aplicar cupón y pagar deshabilitados; subtotal en cero.
 - Intento de agregar más unidades que el stock disponible; producto con stock cero.
 - Cupón inválido: mensaje visible y desglose sin descuento de cupón.
 - Respuestas 400, 404, 409, 500 y fallo de red manejadas por el interceptor.
 - Desglose marcado como desactualizado tras cualquier mutación del carrito.

CRITERIOS TÉCNICOS

- Jest configurado con `jest-preset-angular`; los servicios HTTP se prueban con `HttpTestingController`.
- Mocks de `IProduct`, `ICartItem`, `IDiscountBreakdown` e `IOrder` centralizados en `src/testing/mocks/` con nomenclatura `mock[Entidad]` y barrel `index.ts`.
- Las pruebas de componentes consultan elementos mediante sus atributos `data-test`, no por clases CSS ni estructura del DOM.
- Un `describe` por archivo; textos de `it` en español con estructura "debería [resultado] cuando [condición]".
- Las pruebas del store verifican el valor de los selectores derivados, no solo la invocación de métodos.

CRITERIOS DE SEGURIDAD

- Existe una prueba que verifica que la petición de checkout no incluye precios ni totales en el cuerpo.
- Existe una prueba que verifica que los mensajes de error 500 mostrados al usuario no contienen el texto original del servidor.